this context in the actual instantiation are captured as `Args`. The variadic parameter `Args` is called a *parameter pack*, and the use of `Args...` is called a *pack expansion*.

Just like with variadic functions, the variadic part may be an empty list or may have many entries. For example, with `std::packaged_task<my_class()>` the `ReturnType` parameter is `my_class`, and the `Args` parameter pack is empty, whereas with `std::packaged_task<void(int,double,my_class&,std::string*)>` the `ReturnType` is `void`, and `Args` is the list `int, double, my_class&, std::string*`.

A.6.1. Expanding the parameter pack

The power of variadic templates comes from what you can do with that pack expansion: you aren't limited to just expanding the list of types as is. First off, you can use a pack expansion directly anywhere a list of types is required, such as in the argument list for another template:

```
template<typename ... Params>
struct dummy
{
    std::tuple<Params...> data;
};
```

In this case the single member variable `data` is an instantiation of `std::tuple<>` containing all the types specified, so `dummy<int,double,char>` has a member of type `std::tuple<int,double,char>`. You can combine pack expansions with normal types:

```
template<typename ... Params>
struct dummy2
{
    std::tuple<std::string,Params...> data;
};
```

This time, the tuple has an additional (first) member of type `std::string`. The nifty part is that you can create a pattern with the pack expansion, which is then copied for each element in the expansion. You do this by putting the